Các Thế hệ Ngôn ngữ Lập trình (Language Evolution)



1. Ngôn ngữ máy (Machine Language)

Ngôn ngữ duy nhất phần cứng CPU hiểu và thực thi trực tiếp (Chuỗi số 0 và 1). Phụ thuộc chặt chẽ kiến trúc phần cứng. Cực kỳ khó viết, dễ sai sót và không thể bảo trì.



2. Hợp ngữ (Assembly Language)

Dùng từ gợi nhớ tiếng Anh (LOAD, ADD, STORE). Cần chương trình Assembler dịch sang mã máy. Vẫn phụ thuộc vi xử lý, tốc độ viết code rất chậm.



3. Ngôn ngữ bậc cao (High-Level)

Đại diện: C, C++, Java, C#, Python. Cú pháp gần tiếng Anh và ký hiệu toán học; hoàn toàn độc lập với phần cứng.

Gemini Notebook

Biên dịch (Compiler) vs Thông dịch (Interpreter)

Trình biên dịch (Compiler)



- Dịch toàn bộ mã nguồn thành tệp mã máy trước khi thực thi.
- Tốn thời gian ban đầu nhưng chạy cực kỳ nhanh.
- Báo lỗi cú pháp toàn diện trước khi chạy.
- Đại diện: C, C++, Rust, Go.

Trình thông dịch (Interpreter)



- Đọc và dịch từng dòng lệnh một, chạy trực tiếp rồi mới sang dòng kế.
- Chạy ngay không chờ đợi, tốc độ chậm hơn đáng kể.
- Lỗi dòng nào phát hiện khi chạy đến dòng đó.
- Đại diện: Python, JavaScript, Ruby, PHP.



Cơ chế Lai (Hybrid Approach) trong C# / Java

Biên dịch mã nguồn thành Mã trung gian (MSIL / Bytecode) → Dùng JIT Compiler trong máy ảo dịch sang mã máy siêu tốc ngay khi khởi chạy!

Gemini Notebook

Công nghệ Đối tượng (Object Technology)

Thế giới thực = Các Đối tượng (Objects):
Ô tô, điện thoại, con người, tài khoản.

1. Thuộc tính
(Attributes / Properties)

Màu sắc, kích thước,
số dư tài khoản.



2. Hành vi
(Behaviors / Methods)

Chạy, dừng, tăng tốc,
rút tiền, chuyển khoản.



Mục tiêu Tối thượng của OOP:

Tái sử dụng Phần mềm (Software Reuse)

Thay vì gõ lại từ đầu, sử dụng các lớp đối tượng có sẵn đạt chuẩn chất lượng cao -> Tăng năng suất gấp nhiều lần.

Gemini Notebook

Phân biệt Lớp (Class) và Đối tượng (Object)

Lớp (Class) = Bản vẽ thiết kế



Là khuôn mẫu (Blueprint/Template) định nghĩa cấu trúc dữ liệu và hành vi chung.

Ví dụ: Bản vẽ thiết kế ô tô trên giấy (chỉ định 4 bánh, vô lăng, màu sơn). Bản vẽ không thể tự nổ máy hay chạy trên đường!

Đối tượng (Object) = Thực thể cụ thể



Là một thể hiện cụ thể (Instance) được chế tạo từ bản vẽ lớp.

Ví dụ: Chiếc xe màu đỏ biển 43A-12345 thực sự lăn bánh. Từ 1 Class có thể tạo ra vô số Objects với giá trị thuộc tính riêng!

Cú pháp khởi tạo trong C#

```
Car myCar = new Car("VinFast VF8", "Do"); // Toán tử new sinh đối tượng trong RAM
```

Gemini Notebook

3 Trụ Cột Công Nghệ Đối Tượng Trong C#



1. Đóng gói (Encapsulation)

Gom nhóm dữ liệu và phương thức vào 1 Class.

Che giấu thông tin: Người lái chỉ cần đạp ga, không cần biết chi tiết bơm nhiên liệu!

Bảo vệ dữ liệu nội bộ không bị can thiệp trái phép.



2. Kế thừa (Inheritance)

Tạo Class mới dựa trên Class có sẵn.

Thừa hưởng toàn bộ thuộc tính/hành vi và bổ sung tính năng riêng.

Ví dụ: XeDien kế thừa từ OTo, thêm thuộc tính DungLuongPin.



3. Đa hình (Polymorphism)

Các đối tượng thuộc các lớp khác nhau phản ứng với cùng một thông điệp theo những cách riêng biệt.

Gemini Notebook

Internet và World Wide Web: Các Cột Mốc Tiến Hóa



(1969)
ARPANET: Tiền thân Internet, tiên phong chuyển mạch gói.



Giao thức TCP/IP: Nền tảng kết nối các mạng máy tính.

(1989)
World Wide Web: Tim Berners-Lee phát minh HTML & HTTP.





(2000s)
Web 2.0: Chuyển sang web tương tác, Cloud SaaS.

Cloud Computing: Thuê năng lực tính toán quy mô toàn cầu.





Internet of Things (IoT): Hàng tỷ thiết bị thông minh kết nối.

Gemini Notebook

Ngôn Ngữ C#: Lịch Sử Hình Thành & Chuẩn Hóa

History



.NET Ra mắt năm 2000 bởi Microsoft cùng sáng kiến .NET

Thiết kế bởi Kỹ sư trưởng huyền thoại Anders Hejlsberg

ISO Chuẩn hóa quốc tế bởi ECMA-334 và ISO/IEC 23270

Name Origin



Nguồn gốc tên gọi C# 🎵

Dấu thăng (#) mượn từ âm nhạc mang ý nghĩa
Thăng tiến (Sharp). Ghép từ 4 dấu + của C++ xếp lại.
Biểu thị sự kế thừa hoàn hảo và nâng cấp toàn diện!

Gemini Notebook

Đặc Trưng Cốt Lõi Của C#



Hướng đối tượng 100%
(Pure OOP)



Định kiểu mạnh mẽ
& An toàn-safe)



Thu gom rác tự động
(Garbage Collection)



Hỗ trợ lập trình trực quan
(Visual)



Hỗ trợ xử lý bất đồng bộ
(async/await)



Đa nền tảng
(Cross-platform)

Gemini Notebook

Các Phong Cách Lập Trình Đa Dạng Trong C#

1. Lập trình Hướng sự kiện



Phản hồi các sự kiện do người dùng kích hoạt: Click chuột, gõ gõ phím. Là cốt lõi của các ứng dụng GUI hiện đại.

2. Lập trình Trực quan



Kéo-thả các điều khiển (Button, TextBox) trực tiếp trên Form Designer thay vì ngồi viết code giao diện thủ công.

3. Lập trình Tổng quát



Viết **thuật toán tổng quát** cho mọi kiểu dữ liệu mà vẫn **đảm bảo Type-Safety** (ví dụ: **List<T>**).

4. Lập trình Bất đồng bộ



Thực thi tác vụ **tốn thời gian (I/O, API, CSDL)** mà **không làm đơ/treo giao diện** người dùng **(Async / Await)**.

Gemini Notebook

Nền Tảng Microsoft .NET: Tầm Nhìn Kiến Trúc

Sáng kiến Microsoft .NET (2000)

Nền tảng toàn diện để phát triển, triển khai và thực thi ứng dụng dễ dàng trên mạng.



Thư viện Lớp (.NET FCL)

Hàng chục nghìn lớp tối ưu hiệu năng cao.
Giải pháp trọn gói: File I/O, Mạng, CSDL (ADO.NET), Bảo mật, UI.
Giúp **tiết kiệm 70%** thời gian gõ code.



Máy ảo CLR (Common Language Runtime)

Trái tim vận hành hệ sinh thái .NET.
Ẩn giấu phần cứng & HĐH. Quản lý toàn bộ vòng đời: Cấp phát bộ nhớ, thu gom rác, bảo mật.

Gemini Notebook

Thế Giới Code: Quản Lý vs Không Quản Lý

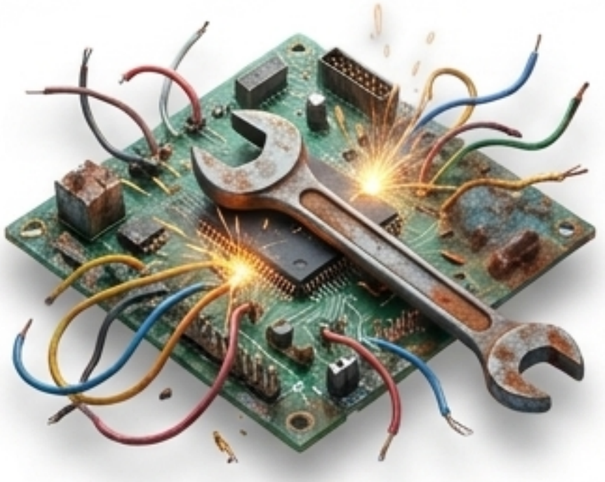
Managed Code (Mã được quản lý)



Biên dịch ra mã trung gian MSIL, chạy dưới sự giám sát hoàn toàn của CLR (C#, VB.NET mặc định).

- ✅ Không rò rỉ bộ nhớ (Memory Leaks).
- ✅ Ngăn ghi đè vùng nhớ (Buffer Overflow).
- ✅ Bắt ngoại lệ an toàn giữa các ngôn ngữ.

Unmanaged Code (Mã không quản lý)



Biên dịch trực tiếp sang mã máy nhị phân CPU (C, C++ truyền thống). Không có CLR giám sát.

- ❌ Lập trình viên tự quản lý từng byte (malloc, free).
- ❌ Nếu lỗi con trỏ có thể làm sập toàn bộ hệ điều hành (Crash / BSoD).

Gemini Notebook

Kiến Trúc Thực Thi .NET: Hành Trình Của Dòng Code

Giai đoạn Biên dịch - Compile Time



Mã nguồn (*.cs)

Trình biên dịch C# (csc.exe)



Mã trung gian MSIL + Metadata (*.dll)

Môi trường Máy ảo CLR - Runtime Execution

CLR JIT Compiler
(Dịch thời gian thực ngay khi chạy)



Mã máy bản địa (Native Code) chạy trên CPU.

Vai trò của MSIL

MSIL không phải mã máy của Intel hay ARM. Tệp *.dll có thể mang sang bất kỳ máy nào (Windows, Linux, macOS) có CLR để chạy mượt mà!

Gemini Notebook

Quyền Năng Vượt Nền Tảng (Platform Independence)

Write Once, Run Anywhere



Viết & biên dịch 1 lần. Mọi HĐH có **CLR** đều chạy được tệp **MSIL** mà không cần sửa hay biên dịch lại.

Tiết kiệm hàng triệu USD chi phí chuyển đổi (porting).

Sự tiến hóa của .NET

2002-2019

.NET Framework

.NET Framework (2002-2019): Tối ưu chủ yếu cho hệ sinh thái Windows.

2016

.NET Core

.NET Core (2016): Cột mốc viết lại hoàn toàn với mã nguồn mở, hoạt động siêu tốc trên Windows, Linux, macOS.

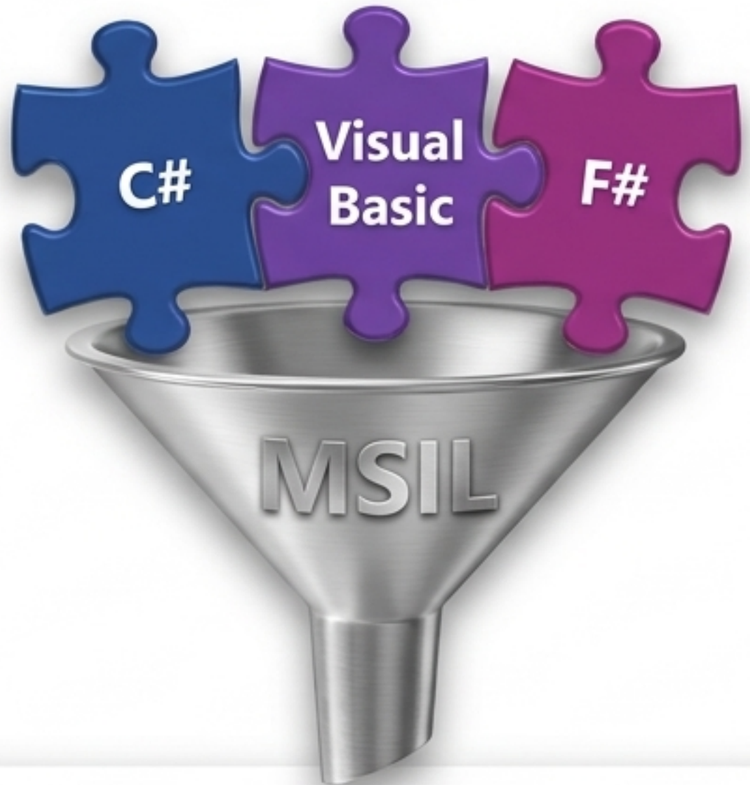
Hiện đại

.NET 5 → 8 → 9

.NET 5 → 8 → 9 (Hiện đại): Hợp nhất toàn bộ hệ sinh thái thành một nền tảng duy nhất, đẳng cấp thế giới.

Gemini Notebook

Tương Tác Đa Ngôn Ngữ (Language Interoperability)



Không bắt buộc dùng 1 ngôn ngữ. Module C#, Module VB, Module F# đều biên dịch về chung một chuẩn mã trung gian!

Sức mạnh Tích hợp Không biên giới:

Lớp viết bằng **C#** có thể **kế thừa** trực tiếp lớp **Visual Basic**.

Phương thức C# có thể **bắt lỗi (Exception)** do mã **F#** ném ra.

Toàn bộ thư viện FCL chia sẻ bình đẳng, dùng chung cho mọi ngôn ngữ trong hệ sinh thái.

Gemini Notebook

[CHECKPOINT 2] & [HANDS-ON LAB 2]

Kiểm tra Nhanh

Câu hỏi 1: Mã nguồn C# biên dịch lần đầu tạo ra mã nào?

-> Đáp án đúng: **Mã trung gian (MSIL / CIL)**

Câu hỏi 2: Thành phần nào dịch MSIL sang mã máy khi chạy?

-> Đáp án đúng: **JIT Compiler**

Thực hành CLI - 10 Phút

```
C:\> dotnet --version
8.0.204

C:\> dotnet new console -o HelloIT26M

C:\> cd HelloIT26M

C:\HelloIT26M> dotnet run
Hello, World!
```

Lệnh **dotnet run** tự động gọi trình biên dịch sinh tệp **HelloIT26M.dll** (chứa MSIL), rồi kích hoạt CLR để chạy!

Gemini Notebook